

Introduction to Data Structures

Data Structures:

A *data structure* is an arrangement of data in a computer's memory or even disk storage.

Data structures can be classified into two types

- Linear Data Structures
- Non Linear Data Structures

Linear Data Structures:

Linear data structures are those data structures in which data elements are accessed (read and written) in sequential fashion (one by one)

Eg: Stacks , Queues, Lists, Arrays

Non Linear Data Structures:

Non Linear Data Structures are those in which data elements are not accessed in sequential fashion.

Eg: trees, graphs

Algorithm:

Step by Step process of representing solution to a problem in words is called an Algorithm.

Characteristics of an Algorithm:

- **Input :** An algorithm should have zero or more inputs
- **Output:** An algorithm should have one or more outputs
- **Finiteness:** Every step in an algorithm should end in finite amount of time
- **Unambiguous:** Each step in an algorithm should clearly stated
- **Effectiveness:** Each step in an algorithm should be effective

Advanced Data Structures

Characteristics of Data Structures

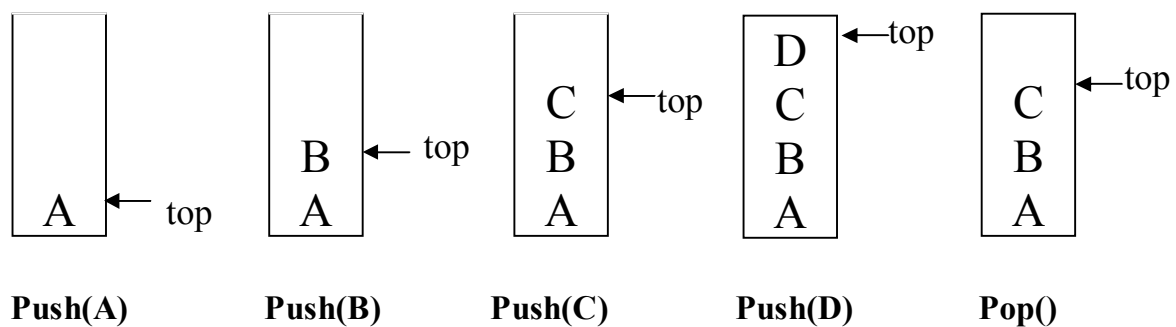
Data Structure	Advantages	Disadvantages
Array	Quick inserts Fast access if index known	Slow search Slow deletes Fixed size
Ordered Array	Faster search than unsorted array	Slow inserts Slow deletes Fixed size
Stack	Last-in, first-out acces	Slow access to other items
Queue	First-in, first-out access	Slow access to other items
Linked List	Quick inserts Quick deletes	Slow search
Binary Tree	Quick search Quick inserts Quick deletes <i>(If the tree remains balanced)</i>	Deletion algorithm is complex

Stack :

Stack is a Linear Data Structure which follows Last in First Out mechanism.

It means: the first element inserted is the last one to be removed

Stack uses a variable called top which points topmost element in the stack. top is incremented while **pushing** (inserting) an element in to the stack and decremented while **poping** (deleting) an element from the stack



Valid Operations on Stack:

- Inserting an element in to the stack (Push)
- Deleting an element in to the stack (Pop)
- Displaying the elements in the queue (Display)

Note:

While pushing an element into the stack, stack is full condition should be checked

While deleting an element from the stack, stack is empty condition should be checked

Applications of Stack:

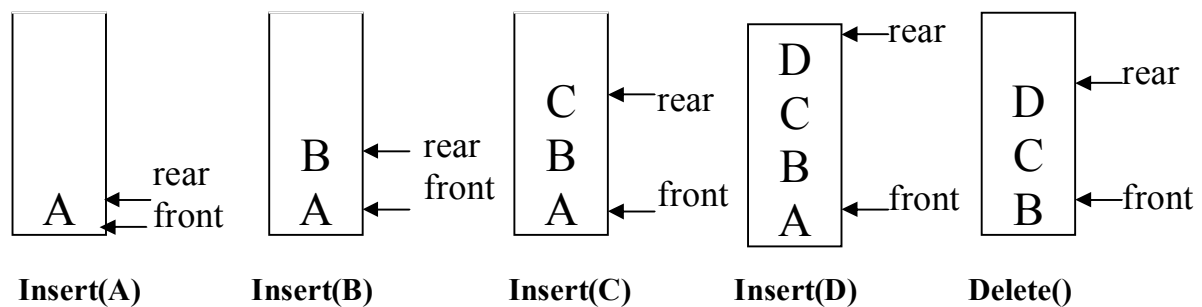
- Stacks are used in recursion programs
- Stacks are used in function calls
- Stacks are used in interrupt implementation

Queue:

Queue is a Linear Data Structure which follows First in First out mechanism.

It means: the first element inserted is the first one to be removed

Queue uses two variables rear and front. Rear is incremented while inserting an element into the queue and front is incremented while deleting element from the queue



Valid Operations on Queue:

- Inserting an element in to the queue
- Deleting an element in to the queue
- Displaying the elements in the queue

Note:

While inserting an element into the queue, queue is full condition should be checked

While deleting an element from the queue, queue is empty condition should be checked

Applications of Queues:

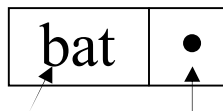
- 🕒 Real life examples
 - 🏠 Waiting in line
 - 🏠 Waiting on hold for tech support
- 🕒 Applications related to Computer Science
 - 🏠 Threads
 - 🏠 Job scheduling (e.g. Round-Robin algorithm for CPU allocation)

Linked List:

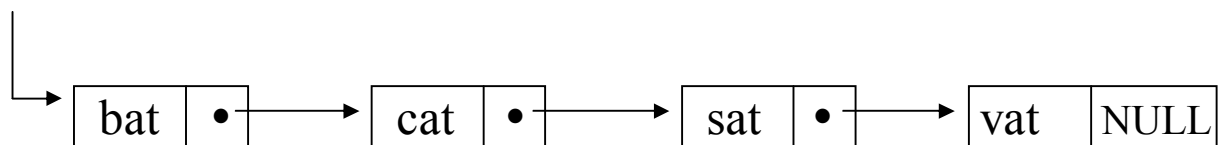
To overcome the disadvantage of fixed size arrays linked list were introduced.

A linked list consists of nodes of data which are connected with each other. Every node consists of two parts: data and the link to other nodes. The nodes are created dynamically.

NODE



Data link



Types of Linked Lists:

- Single linked list
- Double linked list
- Circular linked list

Valid operations on linked list:

- Inserting an element at first position
- Deleting an element at first position
- Inserting an element at end
- Deleting an element at end
- Inserting an element after given element
- Inserting an element before given element
- Deleting given element